System Scanner Pro

AI-Powered Distributed Endpoint Monitoring & Remote Scanning Platform

Technical Documentation Package | v3.1

Project Explanation & Knowledge Transfer

Every module explained: what, why, how, inputs, outputs and failure modes

Project	System Scanner Pro (v3.1)
Document	Project Explanation & Knowledge Transfer
Package date	August 21, 2026
Prepared by	Project Team - Kailasanathan (Team Leader), Saagr, Mega
Audience	Developers, administrators, testers and new team members
Status	Final

This document is part of a nine-volume documentation package generated from the actual project source code. All commands, file paths, endpoints and behaviours described here were verified against the repository at documentation time.

Table of Contents

1. How to Use This Volume

Each section answers the same nine questions for one subsystem: WHAT it is, WHY it exists, HOW it works internally, its INPUTS, PROCESSING steps, OUTPUTS, COMMUNICATION with other parts, FAILURE MODES, and MODIFICATION POINTS. Read the sections in order for a full mental model, or jump to a subsystem when debugging.

2. Module 1 - Authentication & Account Security

Question	Answer
WHAT	Login/logout/session management for dashboard users, plus JWT pairs for API consumers and hashed API keys for integrations.
WHY	Everything else is company-scoped; identity is the root of that scoping.
HOW	Django sessions + signed scanner_auth cookie fallback; PBKDF2 password hashes; lockout via LoginAttempt counting (5 fails / 30 min); every login recorded in LoginHistory.
INPUTS	Username/password POSTs; JWT bearer headers; sk_ API keys; signed cookie.
PROCESSING	Validate -> check lockout -> authenticate -> audit -> issue session/cookie or tokens.
OUTPUTS	Authenticated request.user; session/cookie; JWT pair; ApiKey identity.
COMMUNICATION	All views depend on request.user/company scoping from here.
FAILURE MODES	Locked account (wait or clear LoginAttempt rows); SECRET_KEY rotation invalidating cookies/JWT; clock skew irrelevant (server-side timestamps).
MODIFICATION	scanner_api/views.py auth cluster; jwt_auth.py; api_key_auth.py; middleware.py; validators.py for password policy.

Table 1 - Authentication module brief

3. Module 2 - Company Tenancy & User Onboarding

WHAT - Signup creates a Company + AdministratorProfile; superuser seeding creates the bootstrap tenant.

WHY - Multi-company support keeps customer data isolated without separate deployments.

HOW - get_user_company() resolves the requester's company once per request; querysets filter by it.

INPUTS - Signup form fields; existing user records.

OUTPUTS - Company row + profile link; scoped dashboards.

FAILURE MODES - User without company (legacy rows) falls back to superuser's company or creates one on demand.

MODIFICATION - Company/AdministratorProfile models; get_user_company helper in views.py.

4. Module 3 - Client Registration & Approval Workflow

Question	Answer
WHAT	The gate that turns an unknown machine into a managed device.
WHY	Prevents random machines from injecting telemetry; supports compliance narratives.
HOW	Agent posts registration key + SHA-256 fingerprint. Server matches key; compares fingerprint with stored DeviceFingerprint history; applies demotion rules; sets status pending/approved.
INPUTS	Registration payload: key, hostname, platform, os_version, cpu_model, ram_info, fingerprint, optional admin_username/company_slug from connect URL.
PROCESSING	13 unit-tested scenarios govern outcomes: pending-by-default, same-device re-registration, different-device reset, ping demotion of stale auto-approvals, deleted-client reactivation on approval.
OUTPUTS	Client row (status), ActivityLog entry, dashboard Pending card.
COMMUNICATION	Dashboard polls /api/clients/; WS pushes status changes after approval.
FAILURE MODES	Duplicate keys across machines -> intentional demotion loop until admin resolves; clock skew harmless here.
MODIFICATION	RegisterClientView/PingClientView in views.py; tests/test_registration_approval.py documents intended semantics.

Table 2 - Registration module brief

5. Module 4 - Server Discovery (UDP + Cloud Registry)

WHAT - Mechanisms by which an agent locates the server without manual configuration.

WHY - Zero-touch enrollment at scale; resilience across network topologies.

HOW - Order of attempts: explicit connect URL -> UDP broadcast 'DISCOVER_ADMIN' on port 45000 answered by main.py listener -> Supabase server_registry lookup (published by server startup + hourly GitHub Action).

INPUTS - Broadcast replies; registry REST rows; CLI argument.

OUTPUTS - Base URL used for all subsequent HTTP/WSS calls.

FAILURE MODES - UDP blocked across VLANs (use registry/URL); stale registry row after IP change (Action refreshes hourly; agent retries).

MODIFICATION - client/discovery.py; admin/main.py listener; supabase_client.py; workflow YAML.

6. Module 5 - Agent Scanner Suite

Question	Answer
WHAT	Collectors producing the full machine inventory JSON.
WHY	Inventory is the foundation for diffs, health, assets and reports.
HOW	client/scanner.py enumerates CPU/RAM/storage/GPU/motherboard/network/peripherals; installed software; Windows updates; AV status; local users/administrators/domain info. Windows uses PowerShell/CIM; POSIX paths use /proc, sysctl, dmidecode where available.
INPUTS	OS queries; WMI/CIM results; filesystem probes.
PROCESSING	Normalises vendor quirks into stable keys; attaches collection timestamp/version.
OUTPUTS	Scan payload dict -> communicator upload; local backup copy under client/scans/.
COMMUNICATION	POST submit-scan; monitoring normalises into inventory tables.
FAILURE MODES	PowerShell execution policy blocks (agent invokes with bypass flags); partial data tolerated - missing sections render as empty.
MODIFICATION	Add collectors in client/scanner.py; schema-agnostic storage means new keys appear automatically in detail pages/diffs.

Table 3 - Scanner suite brief

7. Module 6 - Metrics & Heartbeat Telemetry

WHAT - Lightweight live signals distinct from full scans.

HOW - client/metrics.py samples CPU/RAM/disk/net every cycle; heartbeat frame sent every 30 s over WS or REST fallback; server stores DeviceHeartbeat + DeviceMetrics and updates last_seen/status.

WHY - Enables live cards, health scoring and anomaly baselines cheaply.

INPUTS/OUTPUTS - OS counters -> compact JSON frames -> time-series rows.

FAILURE MODES - Missed heartbeats -> stale_checker/offline_detector flip state; bursts queued client-side.

MODIFICATION - Sampling interval constants in config; ingestion in monitoring views/consumers.

8. Module 7 - Event Monitors (USB/File/Process/Software)

Monitor	Trigger cadence	Captures
USB insertion/removal	~5 s poll	Device name/type/serial events
Filesystem changes	watchdog observer	Created/modified/deleted paths in watched roots
Process launches	~10 s diff	New process name/PID/user snapshots
Software installs	~60 s diff	Installed-program list deltas

Table 4 - Event monitor matrix

events/dispatcher.py batches events (max batch size) and ships them via communicator; on send failure events persist to a disk queue (SystemScannerPro/offline_events) and replay on next successful transport - restarts lose nothing.

9. Module 8 - WebSocket Command Channel

WHAT - Persistent agent<->server socket enabling push commands and instant status.

WHY - scan_now responsiveness and live dashboard updates without polling storms.

HOW - AgentConsumer validates HMAC query params before accept; group device_<id> carries command frames; DashboardConsumer mirrors status to browsers.

INPUTS/OUTPUTS - heartbeat_json in; scan_now/out; ack/result frames; dashboard broadcasts.

FAILURE MODES - Proxy stripping Upgrade headers (fix nginx config); Vercel path has no WS by design; reconnect uses exponential backoff with jitter.

MODIFICATION - monitoring/consumers.py + routing; client/communicator.py WS thread.

10. Module 9 - Scan Ingestion, Storage & Diffing

WHAT - Server-side handling of scan payloads.

HOW - SubmitScanView stores raw JSON in ScanResult.scan_data, upserts Hardware/SoftwareInventory rows, runs compute_scan_diff vs previous scan, emits diff to event bus, recomputes health.

WHY - Raw fidelity satisfies audits; normalised tables power fast UI/reports.

INPUTS - Agent scan JSON; scan_type (scheduled/manual/on-demand/local).

OUTPUTS - ScanResult row; inventory upserts; diff record; updated MonitoringInfo.

FAILURE MODES - Malformed JSON rejected with 400; huge payloads capped by server limits - agents chunk if needed.

MODIFICATION - views.SubmitScanView; monitoring/diff_utils.py.

11. Module 10 - Health Scoring Engine

WHAT - Single 0-100 number per device summarising wellbeing.

WHY - Executives need one glance signal; engineers need drill-down components.

HOW - monitoring/health.py weights CPU load, RAM pressure, disk usage, offline recency and alert penalties; component breakdown stored on DeviceMonitoringInfo.

INPUTS - Latest metrics + alerts.

OUTPUTS - health_score + components on device card/detail page; feeds executive dashboard.

FAILURE MODES - Sparse history lowers confidence - engine returns conservative scores rather than errors.

MODIFICATION - Adjust weights table atop health.py.

12. Module 11 - Anomaly Detection Engine

WHAT - Statistical outlier detection on metric series.

METHODS - zscore (deviation from mean), iqr (quartile fences), moving_avg (band around rolling mean), threshold (static limits), compound (consensus of others).

WHY - Catch silent degradations static thresholds miss.

INPUTS - Recent DeviceMetrics windows per metric.

OUTPUTS - Anomaly findings consumed by alert_checker -> Alert rows/notifications.

FAILURE MODES - Noisy environments cause false positives - tune method/sensitivity per rule; cold-start needs N samples before judging.

MODIFICATION - anomaly_detection.py methods registry; rules UI selects method+metric.

13. Module 12 - Predictive Engine

WHAT - Forward-looking estimates: disk-full ETA, failure risk score, capacity forecast.

HOW - Linear regression over usage trend for ETA days; weighted incident factor scoring for risk 0-100; horizon projections for capacity planning.

WHY - Shift ops from reactive tickets to planned maintenance.

OUTPUTS - Predictive alert rows + dashboard badges ('Disk ~14 days to full').

FAILURE MODES - Erratic sampling widens confidence bands - predictions flagged low-confidence instead of alarmist.

MODIFICATION - predictive.py horizons/weights; feature_store.py supplies richer inputs for future ML.

14. Module 13 - Alert Rules & Notification Pipeline

Stage	Behaviour
Rule definition	AlertRule rows: metric/source, condition, severity, anomaly method, notification targets
Evaluation	alert_checker job (300 s) + inline checks after scans; run_alert_checks command for manual runs
Recording	Alert + AlertHistory rows; severity drives colouring/escalation lists
Notification	Notification rows honour NotificationPreference; dashboard bell + WS badge updates
Acknowledgement	Acknowledge/resolve actions append history; audit trail preserved

Table 5 - Alert pipeline stages

15. Module 14 - Asset Lifecycle Management

WHAT - Full procurement-to-retirement tracking with QR labels and documents.

HOW - Asset.status transitions validated against a guarded map (Draft->Pending->Approved->Purchased->Assigned->Retired); assignments v2 track custody; transfers + immutable AssetHistory preserve lineage; AssetDocument stores base64 files; qrcode lib renders label PNGs.

INPUTS - Manual CRUD + Excel import/export (openpyxl); optional linkage from scanned hardware.

OUTPUTS - asset_dashboard analytics (counts by status/category/vendor), asset_detail dossier, executive KPIs.

FAILURE MODES - Invalid transition attempts rejected with explanatory 400; orphan categories pruned manually.

MODIFICATION - scanner_api models (Asset*), assets.js trio, import/export endpoints.

16. Module 15 - Org Structure (Employees/Departments/Locations)

WHAT - HR-style hierarchy giving assets/licenses a custodian context.

HOW - Location -> Department -> Employee chain; EmployeeAssetAssignment bridges to assets; OrgAuditLog records structural edits.

WHY - Compliance reports often need 'who had what, where'.

MODIFICATION - Three models + three JS modules; keep slugs stable - exports reference them.

17. Module 16 - Maintenance Workflow

WHAT - Planned/corrective maintenance records with approvals and paper trail.

HOW - MaintenanceRecord(type, priority, scheduled/completed dates, cost, approval status); MaintenanceHistory appends; documents attach; WarrantyRecord + DowntimeRecord feed impact analysis; maintenance/alerts.py raises upcoming-service alerts.

INPUTS - Manual entries; links to Client or Asset.

OUTPUTS - maintenance.html board, maintenance-alerts queue, downtime minutes per period.

MODIFICATION - maintenance app models/views; alert thresholds in alerts.py.

18. Module 17 - Software License Tracking

WHAT - License inventory with seat assignment and expiry vigilance.

HOW - SoftwareLicense(product, vendor, seats, expiry, cost, masked display key + protected full key); LicenseAssignment binds seats to employees/devices; LicenseHistory logs moves; intelligence alerts warn pre-expiry.

WHY - Avoid surprise true-ups and audits.

MODIFICATION - maintenance models + licenses.html/licenses.js flow.

19. Module 18 - Compliance Tracking & Audit Trails

WHAT - Evidence layer mapping controls to ISO27001/ITIL/SOC2/internal/GDPR frameworks.

HOW - ComplianceRecords hold check/status/evidence links; ComplianceLog appends immutable entries; four audit tables (ActivityLog, AuditLog/OrgAuditLog, intelligence.AuditLogEntry, LoginHistory) capture who-did-what across surfaces; retention policies define cleanup.

OUTPUTS - intelligence/compliance page + audit_logs viewer + exportable packs.

MODIFICATION - intelligence/models.py + audit.py writer helpers.

20. Module 19 - Reporting Engine

WHAT - ~25 report types exported as PDF/Excel/CSV, ad-hoc or scheduled.

HOW - intelligence/reports.py builders query scoped data and render via ReportLab PDF templates or openpyxl workbooks; ScheduledReport rows trigger periodic generation tracked in Report rows.

EXAMPLES - Asset Inventory, Device Health, License Compliance, Audit Summary (the four highlighted in MANUAL_TESTING), plus scan diffs, downtime, warranty expiry packs.

FAILURE MODES - Very large datasets stream row-by-row but may hit serverless timeouts - schedule heavy packs on VPS mode.

MODIFICATION - Add builder function + REPORT_TYPE choice; UI auto-lists it.

21. Module 20 - Executive Dashboard & Analytics Snapshots

WHAT - C-level rollups: fleet health distribution, risk hotspots, asset value, license posture, open alerts trend.

HOW - DashboardAnalytics snapshots cache aggregates; executive-dashboard.js renders Chart.js visuals; WS pushes keep tiles fresh.

MODIFICATION - Snapshot writer in intelligence; tile configs in JS module.

22. Module 21 - Settings & Configuration Surface

WHAT - Runtime-tunable behaviour without redeploys (Setting key/value store + settings.html UI).

INCLUDES - Company profile, scan defaults/intervals, auto-approve flag (currently forced off in code), API key management, password policy messaging, reset token parking.

MODIFICATION - Setting.get/set helpers; add keys + form fields in settings view/JS.

23. Module 22 - Deployment Subsystems

Path	Explanation
admin/main.py	Universal launcher: migrate -> seed -> scheduler -> self-client -> UDP listener -> serve (HTTP/Daphne)
deploy/install.sh	Idempotent Debian/Ubuntu provisioning ending in running systemd service + TLS
deploy/nginx/scanner.conf	Reverse proxy incl. WebSocket upgrade + ACME webroot
deploy/systemd/*.service	Unit running launcher with --asgi --domain
api/index.py + vercel.json	Serverless wrapper: gunicorn WSGI, whitenoise, signed cookies, guarded /__reset/
.github/workflows/supabase-register.yml	Hourly discovery keep-alive publishing ADMIN_SERVER_URL

Table 6 - Deployment artefacts

24. Module 23 - Logging & Observability

SERVER - Console INFO + admin/local_server.log(.err) tee; named loggers scanner_api/monitoring/channels/daphne.

AGENT - console prints with timestamps; --rescue writes diagnostics bundle; crashes append client_crash.log.

AUDIT - Business-level observability lives in DB tables (see Module 18) - query them before grepping logs.

25. Module 24 - Cross-Cutting Conventions

- All list APIs accept ?company= scoping implicitly from requester; never trust client-supplied company ids.
- JSON responses follow {status: ok|error, message/data}; HTTP codes stay semantic.
- Timestamps UTC in DB, rendered local in templates.
- Soft deletes only on Client; other entities hard-delete via UI with confirmation modals.

26. Module 25 - Failure Mode Playbook

Symptom	First suspects	Fast check
Agent stuck Pending forever	Fingerprint changed / auto-approve off	Compare fingerprints on card; approve manually
Device shows offline though running	Heartbeat blocked; stale detector	Check agent logs for send failures; firewall
No live updates	WS not connected	VPS mode? nginx upgrade headers? console errors
Report generation timeout	Serverless 30 s cap	Run on VPS or reduce range
Lockouts en masse	Shared account hammered	Inspect LoginAttempt IPs; rotate credentials
Anomaly storm after rollout	Cold baseline + noisy metric	Raise sensitivity threshold temporarily

Table 7 - Quick triage table

27. Module 26 - Where Knowledge Lives (Map of Truth)

Topic	Authoritative source
Intended approval semantics	tests/test_registration_approval.py (read like a spec)
Auth guarantees	jwt_auth.py / api_key_auth.py / middleware.py / monitoring/security.py
Scheduler truth	monitoring/scheduler.py + management/commands/
API surface	urls.py files (four apps)
Deployment reality	deploy/ tree + vercel.json + api/index.py
Test expectations	MANUAL_TESTING.txt (manual) + tests/ (automated)

Table 8 - Source-of-truth index